Master DSA Patterns - Short Notes

50 LeetCode problems, 10 patterns. Quick revision sheet.

1. Two Pointers

Use when: Sorted array / pairs / shrink from both ends. O(n) instead of $O(n^2)$.

- **11 Container With Most Water** - move the shorter wall inward; area = min(h) x width.
- **15 3Sum** - sort, fix i, two pointers for the rest; skip duplicates.
- **167 Two Sum II** - sum too small: left++, too big: right--.

2. Sliding Window

Use when: Contiguous subarray/substring with a condition. Expand right, shrink left.

- **3 Longest Substring No Repeat** - map char -> last index; jump left past the duplicate.
- **76 Minimum Window Substring** - need[] counts + missing counter; shrink while valid.
- **209 Min Size Subarray Sum** - positives only; shrink while sum >= target.

3. Binary Search

Use when: Sorted data or a yes/no monotonic condition. O(log n). mid = l + (r-l)/2.

- **704 Binary Search** - base template (l <= r).
- **35 Search Insert Position** - lower bound: first index with value >= target.
- **34 First and Last Position** - lowerBound(t) and lowerBound(t+1) - 1.
- **33 Rotated Sorted Array** - one half is always sorted; check if target is in it.

4. Hash Map / Set

Use when: Need fast lookup, counting, or duplicate checks. O(1) average.

- **1 Two Sum** - store value -> index, look up target - x.
- **49 Group Anagrams** - key = 26-letter count (or sorted string).
- **128 Longest Consecutive Seq** - set; start only where n-1 is absent.
- **217 Contains Duplicate** - set.add() returns false on repeat.
- **242 Valid Anagram** - +1 for s, -1 for t; all counts must be 0.

5. Prefix Sum

Use when: Range sums / subarray sum counts. sum(l..r) = P[r+1] - P[l].

- **303 Range Sum Query** - precompute prefix array; O(1) per query.
- **560 Subarray Sum Equals K** - map of prefix counts; add count of (sum - k); seed {0:1}.
- **724 Find Pivot Index** - left == total - left - nums[i].
- **974 Subarray Sums Div by K** - count equal prefix remainders; fix negatives with ((x%k)+k)%k.
- **1480 Running Sum** - in-place prefix sum.

6. Stack / Monotonic Stack

Use when: Matching pairs, 'next greater/smaller', histogram problems. Each item pushed/popped once.

- **20 Valid Parentheses** - push the expected closer; pop must match.
- **84 Largest Rectangle Histogram** - increasing stack of indices; on a shorter bar pop and compute area; sentinel 0 at end.
- **155 Min Stack** - second stack holding the running minimum.
- **496 Next Greater Element I** - decreasing stack over nums2 + map.
- **739 Daily Temperatures** - decreasing stack of indices; answer = i - popped.

7. Heap / Priority Queue

Use when: Top-K, k-way merge, streaming median. Use the opposite heap: min-heap for K largest.

- **23 Merge k Sorted Lists** - min-heap of list heads. O(N log k).
- **215 Kth Largest** - min-heap of size k; root is the answer.
- **295 Median from Data Stream** - max-heap (lower half) + min-heap (upper half), keep sizes balanced.
- **347 Top K Frequent** - frequency map + min-heap of size k (or bucket sort).
- **973 K Closest Points** - max-heap of size k by squared distance.

8. BFS / DFS

Use when: BFS (queue): levels, shortest path. DFS: components, flood fill. Always mark visited.

- **102 Level Order Traversal** - queue; process queue.size() nodes per level.
- **127 Word Ladder** - BFS; try 26 letters per position; remove from set = visited.
- **994 Rotting Oranges** - multi-source BFS; each level = 1 minute; check fresh == 0.
- **200 Number of Islands** - DFS from each '1', sink the island to '0'.
- **547 Number of Provinces** - count connected components with DFS.

9. Dynamic Programming

Use when: Overlapping subproblems. Define state, recurrence, base case, then optimise space.

- **70 Climbing Stairs** - f(n) = f(n-1) + f(n-2).
- **198 House Robber** - best = max(skip, take + nums[i]).
- **300 Longest Increasing Subseq** - $O(n^2)$ dp, or tails[] + binary search for O(n log n).
- **322 Coin Change** - dp[a] = min(dp[a-coin] + 1); infinity = amount+1.
- **1143 Longest Common Subseq** - equal chars: dp[i-1][j-1] + 1; else max(up, left).

10. Greedy

Use when: Local best choice leads to global best. Often sort first. (Numbers partly hidden in the image.)

- **45 Jump Game II** - BFS-style: curEnd / farthest, jump when i == curEnd.
- **55 Jump Game** - track farthest reach; fail if i > farthest.
- **134 Gas Station** - if total < 0 then -1; reset start when the tank goes negative.
- **435 Non-overlapping Intervals** - sort by end time; keep earliest-ending; removed = n - kept.
- **763 Partition Labels** - last index per letter; cut when i == current end.

Study order

Hash Map, Two Pointers, Sliding Window, Binary Search, Stack, BFS/DFS, Heap, DP, Greedy.